

BACKEND IMPLEMENTATIONS

Build.gradle.kts

```
plugins {  
    id("com.android.application")  
    id("org.jetbrains.kotlin.android")  
}  
  
android {  
    namespace = "com.example.fraudbot"  
    compileSdk = 36 // Required to fix the "AAR metadata" errors  
  
    defaultConfig {  
        applicationId = "com.example.fraudbot"  
        minSdk = 24  
        targetSdk = 36  
        versionCode = 1  
        versionName = "1.0"  
  
        testInstrumentationRunner =  
"androidx.test.runner.AndroidJUnitRunner"  
    }  
  
    buildTypes {  
        release {  
            isMinifyEnabled = false  
            proguardFiles(  
getDefaultProguardFile("proguard-android-optimize.txt"),  
                "proguard-rules.pro"  
            )  
        }  
    }  
  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_1_8  
        targetCompatibility = JavaVersion.VERSION_1_8  
    }  
}
```

```

    }
    kotlinOptions {
        jvmTarget = "1.8"
    }
    buildFeatures {
        mlModelBinding = true
    }
}

dependencies {
    // Stable versions to prevent "Unresolved reference" errors
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")

    implementation("com.google.android.material:material:1.11.0")

    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    // AI and Translation Libraries
    implementation("com.google.mlkit:translate:17.0.2")

    implementation("org.tensorflow:tensorflow-lite-support:0.4.4")

    implementation("org.tensorflow:tensorflow-lite-metadata:0.4.4")
)

    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")

    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}

```

Activity main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <TextView
            android:id="@+id/chatDisplay"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="16sp"
            android:text="@string/bot_greeting" />
        </ScrollView>

        <EditText
            android:id="@+id/userInput"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="@string/type_hint"
            android:inputType="textLongMessage"
            android:importantForAutofill="no" />

        <Spinner
            android:id="@+id/languageSpinner"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="10dp" />
```

```
<Button
    android:id="@+id/sendButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="@string/btn_text" />
```

```
</LinearLayout>
```

MainActivity.kt

```
package com.example.fraudbot
```

```
import android.Manifest
import android.content.pm.PackageManager
import android.os.Bundle
import android.speech.tts.TextToSpeech
import android.speech.tts.TextToSpeech.QUEUE_FLUSH
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import com.google.mlkit.common.model.DownloadConditions
import com.google.mlkit.nl.translate.TranslateLanguage
import com.google.mlkit.nl.translate.Translation
import com.google.mlkit.nl.translate.TranslatorOptions
import java.util.Locale
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var tts: TextToSpeech
    private lateinit var chatDisplay: TextView
    private lateinit var userInput: EditText
    private lateinit var sendButton: Button
    private lateinit var languageSpinner: Spinner
```

```
    private val languageMap = mapOf(
```

```

        "Hindi" to TranslateLanguage.HINDI,
        "Bengali" to TranslateLanguage.BENGALI,
        "Telugu" to TranslateLanguage.TELUGU,
        "Marathi" to TranslateLanguage.MARATHI,
        "Tamil" to TranslateLanguage.TAMIL,
        "Gujarati" to TranslateLanguage.GUJARATI,
        "Kannada" to TranslateLanguage.KANNADA
    )

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Request SMS Permissions immediately
        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.RECEIVE_SMS) !=
PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
arrayOf(Manifest.permission.RECEIVE_SMS,
Manifest.permission.READ_SMS), 1)
        }

        chatDisplay = findViewById(R.id.chatDisplay)
        userInput = findViewById(R.id.userInput)
        sendButton = findViewById(R.id.sendButton)
        languageSpinner = findViewById(R.id.languageSpinner)

        val adapter = ArrayAdapter(this,
android.R.layout.simple_spinner_dropdown_item,
languageMap.keys.toList())
        languageSpinner.adapter = adapter

        tts = TextToSpeech(this) { status ->
            if (status != TextToSpeech.ERROR) {
                // FIXED: Rectified the Locale warning
                tts.language = Locale.forLanguageTag("hi-IN")
            }
        }
    }

```

```

        sendButton.setOnClickListener {
            val message = userInput.text.toString().trim()
            if (message.isNotEmpty()) {
                chatDisplay.append("\n\nUser: $message")
                val selectedLang =
                    languageSpinner.selectedItem.toString()
                val targetCode = languageMap[selectedLang] ?:
                    TranslateLanguage.HINDI
                processAiLogic(message, targetCode)
                userInput.text.clear()
            }
        }
    }

    private fun processAiLogic(userText: String, targetLang:
String) {
        val isFraud = userText.contains("otp", true) ||
            userText.contains("password", true) ||
            userText.contains("bank", true)

        val options = TranslatorOptions.Builder()
            .setSourceLanguage(TranslateLanguage.ENGLISH)
            .setTargetLanguage(targetLang)
            .build()

        val translator = Translation.getClient(options)
        chatDisplay.append("\nBot: Analyzing...")

        translator.downloadModelIfNeeded(DownloadConditions.Builder().
            build())

            .addOnSuccessListener {
                translator.translate(userText)
                    .addOnSuccessListener { translatedText ->
                        val prefix = if (isFraud) "⚠️ FRAUD
ALERT: " else "✅ SAFE: "
                        val finalMsg = "$prefix$translatedText"

```

```

        chatDisplay.append("\nBot: $finalMsg")
        speakText(finalMsg)
    }
}

private fun speakText(text: String) {
    tts.speak(text, QUEUE_FLUSH, null, null)
}

override fun onDestroy() {
    if (::tts.isInitialized) {
        tts.stop()
        tts.shutdown()
    }
    super.onDestroy()
}
}

```

Strings.xml

```

<resources>
    <string name="app_name">Fraud Bot</string>
    <string name="bot_greeting">Chatbot: Hello! I am your
Guardian.</string>
    <string name="type_hint">Type message here...</string>
    <string name="btn_text">Check Fraud &
Translate</string>
</resources>

```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission
  android:name="android.permission.RECEIVE_SMS" />
    <uses-permission android:name="android.permission.READ_SMS"
  />

    <uses-feature android:name="android.hardware.telephony"
  android:required="false" />
    <uses-permission
  android:name="android.permission.SYSTEM_ALERT_WINDOW" />

    <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportsRtl="true"

  android:theme="@style/Theme.AppCompat.Light.NoActionBar">

      <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
          <action
  android:name="android.intent.action.MAIN" />
          <category
  android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>

      <service
```

```

        android:name=".FloatingBotService"

        android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE"

        android:exported="true">
        <intent-filter>
            <action
android:name="android.view.accessibility.AccessibilityService"
/>

        </intent-filter>
        <meta-data
            android:name="android.accessibilityservice"

        android:resource="@xml/accessibility_service_config" />
        </service>

    </application>
</manifest>

```

SmsReceiver.kt

```

package com.example.fraudbot
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.provider.Telephony
import android.widget.Toast

class SmsReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action ==
Telephony.Sms.Intents.SMS_RECEIVED_ACTION) {
            val messages =
Telephony.Sms.Intents.getMessagesFromIntent(intent)
            for (sms in messages) {
                val body = sms.displayMessageBody

                // Automatic logic: detects fraud without typing
                val isFraud = body.contains("otp", true) ||

```

```

        body.contains("bank", true) ||
        body.contains("password", true)

        if (isFraud) {
            // This shows a warning immediately on your
screen
            Toast.makeText(context, "🚨 SECURITY ALERT:
Potential Fraud SMS detected!", Toast.LENGTH_LONG).show()
        }
    }
}
}
}
}

```

[FraudAccessibilityService.kt](#)

```
package com.example.fraudbot
```

```

import android.accessibilityservice.AccessibilityService
import android.view.accessibility.AccessibilityEvent
import android.view.accessibility.AccessibilityNodeInfo
import android.widget.Toast

class FraudAccessibilityService : AccessibilityService() {
    override fun onAccessibilityEvent(event:
AccessibilityEvent) {
        val rootNode = rootInActiveWindow ?: return
        checkNodes(rootNode)
    }

    private fun checkNodes(node: AccessibilityNodeInfo) {
        val content = node.text?.toString()?.lowercase() ?: ""
        if (content.contains("otp") || content.contains("bank")
|| content.contains("blocked")) {
            Toast.makeText(this, "⚠️ FRAUD DETECTED IN
MESSAGE!", Toast.LENGTH_LONG).show()
        }
        for (i in 0 until node.childCount) {
            node.getChild(i)?.let { checkNodes(it) }
        }
    }
}

```

```

    }
}

    override fun onInterrupt() {}
}

```

Accessibility_service_config

```

<?xml version="1.0" encoding="utf-8"?>
<accessibility-service
xmlns:android="http://schemas.android.com/apk/res/android"

android:accessibilityEventTypes="typeWindowStateChanged|typeWindowContentChanged"
    android:accessibilityFeedbackType="feedbackSpoken"

android:accessibilityFlags="flagDefault|flagIncludeNotImportantViews|flagRetrieveInteractiveWindows"
    android:canRetrieveWindowContent="true"
    android:description="@string/app_name"

android:packageNames="com.google.android.apps.messaging,com.samsung.android.messaging" />

```

FloatingBotService.kt

```

package com.example.fraudbot
import android.accessibilityservice.AccessibilityService
import android.graphics.PixelFormat
import android.view.*
import android.view.accessibility.AccessibilityEvent
import android.view.accessibility.AccessibilityNodeInfo
import android.widget.Button
import android.speech.tts.TextToSpeech
import java.util.Locale

class FloatingBotService : AccessibilityService() {
    private lateinit var windowManager: WindowManager
    private lateinit var floatingButton: Button
    private lateinit var tts: TextToSpeech

```

```

        override fun onServiceConnected() {
            windowManager = getSystemService(WINDOW_SERVICE) as
WindowManager
            tts = TextToSpeech(this) { tts.language =
Locale.forLanguageTag("hi-IN") }

            floatingButton = Button(this).apply {
                text = "🔊 Read Screen"
                setBackgroundColor(android.graphics.Color.RED)
                setTextColor(android.graphics.Color.WHITE)
            }

            val params = WindowManager.LayoutParams(
                WindowManager.LayoutParams.WRAP_CONTENT,
                WindowManager.LayoutParams.WRAP_CONTENT,
WindowManager.LayoutParams.TYPE_ACCESSIBILITY_OVERLAY,
                WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE,
                PixelFormat.TRANSLUCENT
            ).apply {
                gravity = Gravity.TOP or Gravity.START
                x = 100
                y = 100
            }

            floatingButton.setOnClickListener {
                val rootNode = rootInActiveWindow
                if (rootNode != null) {
                    readNode(rootNode)
                }
            }

            windowManager.addView(floatingButton, params)
        }

        private fun readNode(node: AccessibilityNodeInfo) {
            val content = node.text?.toString() ?: ""

```

```
        if (content.isNotEmpty()) {
            val prefix = if
(content.toLowerCase().contains("otp")) "Warning: " else ""
            tts.speak("$prefix$content",
TextToSpeech.QUEUE_FLUSH, null, null)
        }
        for (i in 0 until node.childCount) {
            node.getChild(i)?.let { readNode(it) }
        }
    }

    override fun onAccessibilityEvent(event:
AccessibilityEvent) {}
    override fun onInterrupt() {}
}
```